

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA
ENGENHARIA DE SOFTWARE - TURMA 1ESR



EQUIPE DAC:

ANDRÉ VICTOR GONÇALVES NASCIMENTO - RM 570567

DAVI DIAS DE SOUZA FREITAS - RM 574089

DAVID MIKAEL DIAS DA SILVA - RM 571637

GABRIEL NOVAGA PEREIRA - RM 573196

MATHEUS MONTEIRO DA SILVA - RM 573842

SPRINT 2 - COMPUTATIONAL THINKING WITH PYTHON
DOCUMENTAÇÃO: SISTEMA DE ANÁLISE E ORGANIZAÇÃO DE FOTOS

SÃO PAULO, SP
2026

SUMÁRIO

1. INTRODUÇÃO.....	2
2. JUSTIFICATIVA.....	2
3. LISTAGEM DE FUNCIONALIDADES.....	4
4. INSTRUÇÕES DE EXECUÇÃO.....	4
4.1 Sem Docker no Windows.....	4
4.2 Com Docker.....	5
5. CONCLUSÃO.....	5

1. INTRODUÇÃO

Esta documentação apresenta a especificação técnica do projeto desenvolvido para a Sprint 2 de Computational Thinking With Python e instruções de execução do programa. O objetivo desta etapa de desenvolvimento do projeto é aplicar algumas das principais funcionalidades da câmera que o grupo DAC propôs à empresa Jovi, com o objetivo de atrair mais estudantes a utilizarem smartphones fabricados pela empresa e seu sistema de câmeras. Para implementar tais funcionalidades, o grupo optou por realizar um menu no terminal que fornece opções de pré-processamento e organização de imagens que estarão presentes no sistema final.

O projeto foi desenvolvido utilizando a linguagem Python, nos ambientes de desenvolvimento PyCharm e Visual Studio Code, pois são as IDEs mais utilizadas nas aulas, e buscou seguir as instruções enviadas inicialmente pelos professores. As principais instruções para este trabalho foram:

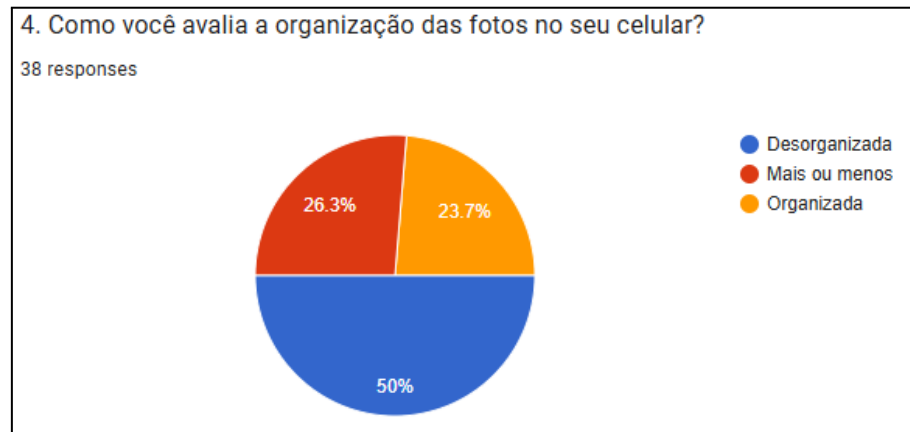
- Implementar um menu de opções com, pelo menos, 3 funcionalidades relevantes do projeto;
- Após a execução de cada funcionalidade, o programa deve retornar para o menu principal;
- O programa deve disponibilizar uma opção para finalizar a execução;

Por fim, o grupo também visou implementar os conceitos aprendidos nas aulas, como a manipulação de variáveis, a entrada e saída de dados, as estruturas de repetição e decisão, a inserção de comentários para ilustrar o funcionamento do código e a organização. A documentação trata de tópicos como a justificativa do projeto, a listagem de funcionalidades e as instruções de execução.

2. JUSTIFICATIVA

Para desenvolver uma proposta de solução que atendesse as dores dos estudantes usuários das câmeras de smartphones da JOVI, o grupo focou em dois dos principais problemas encontrados: a organização de fotos na galeria e a melhoria de reconhecimento e contraste de texto. A decisão de focar nessas dores veio de uma pesquisa realizada pelo grupo, cujo objetivo era entender quais eram as principais dificuldades enfrentadas por estudantes ao utilizarem a câmera dos celulares para estudos. A pesquisa conduzida mostrou que muitos estudantes consideram suas galerias desorganizadas, como mostra o gráfico da Figura 1, além de a utilizarem para capturar fotos que contém texto, como lousas, slides e anotações, como mostra a Figura 2.

FIGURA 1 - Gráfico de respostas sobre organização de fotos



Fonte: Autoria própria, 2026

FIGURA 2 - Gráfico de respostas sobre tipos de conteúdos escritos fotografados



Fonte: Autoria própria, 2026

Para solucionar tais dores, o grupo propôs a utilização de ferramentas para identificar textos nas imagens e aumentar o contraste automaticamente, como o OpenCV, em Python, visando uma melhor visualização dos textos escritos. O grupo também optou pela utilização de ferramentas de IA disponibilizadas pela API do Google Cloud para reconhecer o conteúdo do texto presente na imagem, e criar tags à partir dos textos identificados, para separar fotos por matérias. Após a criação das tags, elas serão adicionadas como prefixos nos nomes das imagens e, por meio delas, as imagens serão guardadas de forma automática em pastas separadas por matérias, que levam o mesmo nome das tags, facilitando a procura de conteúdos na galeria.

Outras funções disponíveis para o usuário serão um plano de estudos feito pela IA ao reconhecer o conteúdo da imagem, a customização de tags criadas, a criação de novas tags manualmente e o recorte de fotos recém tiradas, sem a necessidade de abrir a galeria para isso. Por fim, o diferencial do projeto se dá na organização automática de fotos baseada em conteúdos, no reconhecimento de texto por IA, facilitado pelo aumento de contraste, e na integração de um modo focado em estudantes na câmera.

3. LISTAGEM DE FUNCIONALIDADES

Nesta etapa do projeto, o grupo desenvolveu um sistema com as seguintes funcionalidades disponíveis:

- Aumento de contraste automático de fotos, feito pelo OpenCV;
- Geração de tags com base no conteúdo da imagem, limitadas em “Estudo”, “Trabalho”, “Pessoa”, “Pet” e “Outros”, realizada através da Google Cloud AI;
- Separação das imagens processadas em pastas destinadas às tags específicas;
- Caso a imagem seja classificada como “Estudo”, a Google Cloud AI cria um plano de estudos para tal matéria;
- Após todo o processo, pastas são criadas para armazenar os planos de estudos, os arquivos .json recebidos pela IA, as imagens processadas e as cópias das imagens originais;
- Opção de processar todo o diretório pré-definido de imagens (./imagens_iniciais);
- Opção de processar apenas uma imagem presente no diretório, escolhida pelo usuário;
- Opção de encerrar o sistema e interromper processamento.

4. INSTRUÇÕES DE EXECUÇÃO

Essa seção aborda instruções de execução do projeto, que pode ser executado com e sem utilizar o Docker.

4.1 Sem Docker no Windows

De forma OPCIONAL, pode-se criar um ambiente virtual, utilizando os seguintes comandos no CMD, estando no diretório do projeto:

- `python -m venv .venv`
- `.venv\Scripts\activate`

Para executar o programa, são necessárias as seguintes bibliotecas externas:

- OpenCV;
- Pillow;
- NumPy;
- python-dotenv;
- Google GenAI SDK.

Para a instalação das bibliotecas acima, é necessário ter o Python instalado no computador e executar o seguinte comando no CMD:

- `pip install opencv-python pillow numpy python-dotenv google-genai`

Ou, caso queira instalá-las individualmente:

- `pip install opencv-python`
- `pip install pillow`
- `pip install numpy`

- pip install python-dotenv
- pip install google-generativeai

É preciso criar um arquivo .env e inserir uma API Key do Gemini, na seguinte formatação:

- GEMINI_API_KEY=sua_chave_aqui

Por fim, para inserir imagens novas no sistema, basta adicioná-las na pasta “imagens_iniciais”, encontrada na raiz do projeto, e para executar o programa, o CMD deve se encontrar no diretório onde está o projeto e qualquer um dos seguintes comandos deve ser escrito:

- python [main.py](#)
- py [main.py](#)

4.2 Com Docker

Primeiro, é necessário baixar o Docker Desktop, disponível no link abaixo, e executá-lo.

- <https://www.docker.com/>

É preciso criar um arquivo .env e inserir uma API Key do Gemini, na seguinte formatação:

- GEMINI_API_KEY=sua_chave_aqui

Para executar o programa, é necessário abrir o terminal na pasta do projeto, através do comando cd.

- cd C:\Seu\Diretório

No terminal, o comando de build da imagem do docker deve ser escrito:

- docker build -t challenge-jovi .

Por fim, o projeto é executado através deste comando:

- docker run -it --rm ^ --env-file .env ^ -v
"%cd%/imagens_iniciais:/app/imagens_iniciais" ^ -v
"%cd%/resultados:/app/resultados" ^ challenge-jovi

5. CONCLUSÃO

O grupo foi capaz de concluir todos os requisitos propostos para o trabalho, sendo esses a criação de um menu de opções para o usuário, a implementação de pelo menos 3 funções do sistema final, como o aumento de contraste, criação de tags, organização automática das fotos e criação de planos de estudos, o sistema retorna para o menu inicial

após o fim de cada funcionalidade e fornece a opção de interromper processamento e finalizar execução.